Project - High Level Design

on

Deploy Hospitality App to Kubernetes

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr No	Student Name	Enrolment
1	Irifa Khaishagi	EN22CS301435
2	Aastha Gupta	EN22CS301019
3	Akash Ghosh	EN22CS301082
4	Aman Gour	EN22CS301104
5	Aeshna Preyasi	EN22CS301075
6	Adarsh Dibey	EN22CS301037

Group Name: Group 08D1

Project Number: DO-08

Industry Mentor Name: Aashruti Shah (Program Director, Datagami)

University Mentor Name: Dr. Ritesh Joshi

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. System Architecture Design
   2.2. Application Design
   2.3. Process Flow.
   2. 4. Information Flow.
   2.5. Components Design
   2.6. Key Design Considerations
   2.7 API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

1.1 Scope of the Document

This document describes the high-level design of the Dockerized Multi-Agent Hospitality System deployed on Kubernetes. It explains the overall architecture, component interaction, request flow, data handling, and non-functional aspects of the system. The design keeps the application stateless and containerized, with external API-based intelligence and Kubernetes-based deployment.

1.2 Intended Audience

This document is intended for developers, DevOps engineers, testers, and faculty evaluators.

1.3 System Overview

The system is an AI-powered travel planning web application that accepts user inputs such as destination, duration, and budget and generates a structured itinerary. The application uses CrewAI to manage a sequential multi-agent workflow consisting of Research, Planner, Budget, Writer, and Reviewer agents. The system is containerized and deployed on Kubernetes, with Groq LLM API and DuckDuckGo Search API used as external services.

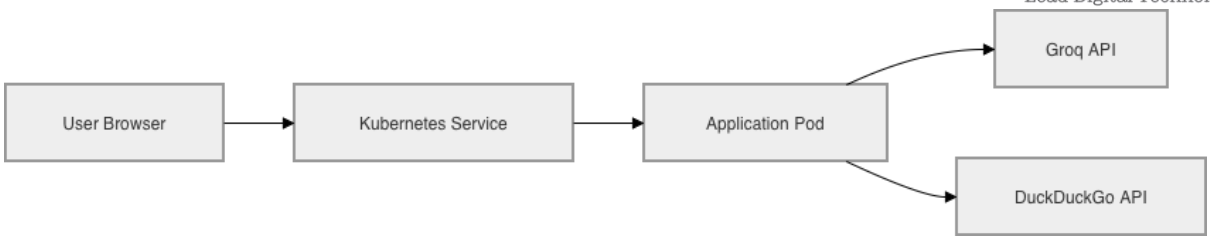
2. System Design

2.1 System Architecture Design

The system architecture shows the overall deployment structure of the Dockerized Multi-Agent Hospitality System on Kubernetes. The application is packaged as a container and runs inside a Kubernetes Pod. A Kubernetes Service is used to expose the application to the user.

The user interacts with the system through the Streamlit web interface. Once the request is submitted, the application processes it inside the pod using the CrewAI-based backend logic. The system then connects to external services such as the Groq LLM API and DuckDuckGo Search API to generate the final itinerary. The architecture remains stateless and does not use a database.

Diagram 1: System Architecture Design



2.2 Application Design

The application follows a simple containerized architecture rather than a database-driven microservices model. The full application runs inside a container and is deployed as a pod in Kubernetes. The design contains three main logical layers:

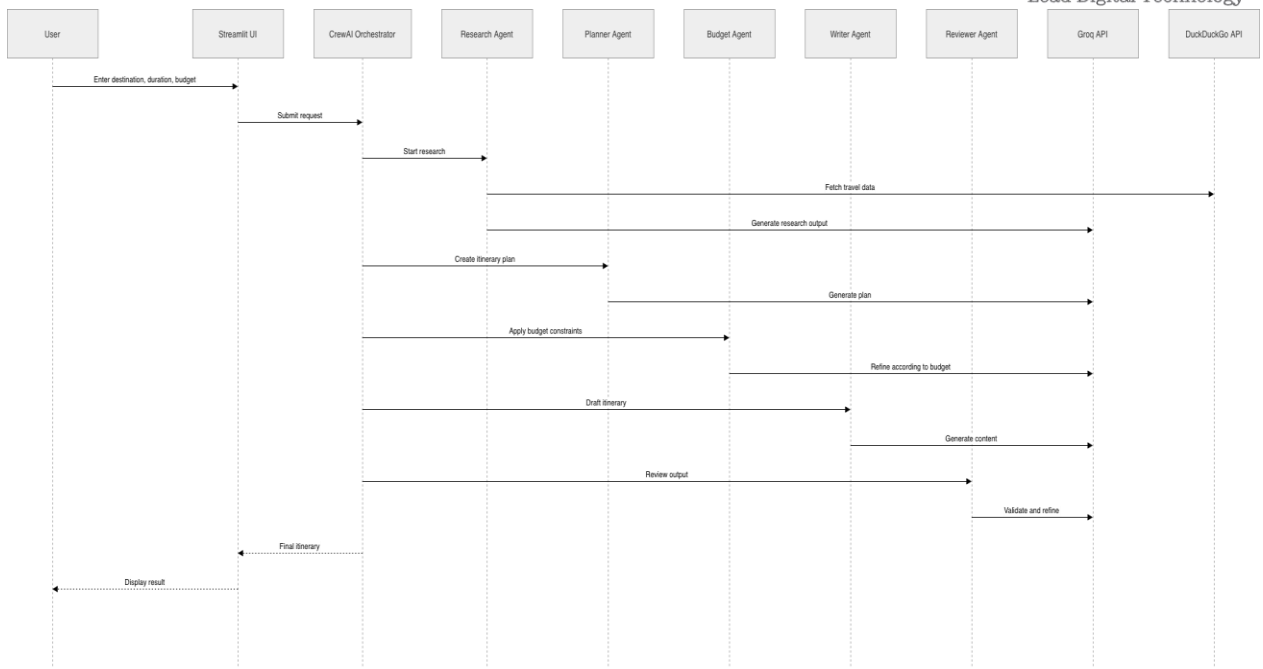
- **Presentation Layer:** Streamlit UI for user input and result display
- **Business Logic Layer:** CrewAI orchestrator for workflow control
- **Agent Layer:** Research, Planner, Budget, Writer, and Reviewer agents

The system does not use an internal REST backend. Instead, the orchestration happens internally, and external APIs are called directly where needed.

2.3 Process Flow

1. The user opens the Streamlit web interface.
2. The user enters travel preferences such as destination, duration, and budget.
3. The application sends the request to the CrewAI orchestrator.
4. The orchestrator triggers the Research Agent first.
5. The Research Agent fetches travel-related information using DuckDuckGo and Groq.
6. The Planner Agent creates the travel plan.
7. The Budget Agent adjusts the output according to budget constraints.
8. The Writer Agent prepares the final itinerary text.
9. The Reviewer Agent validates and refines the response.
10. The final itinerary is displayed back in the UI.

Diagram 2: Process Flow Diagram



2.4 Information Flow

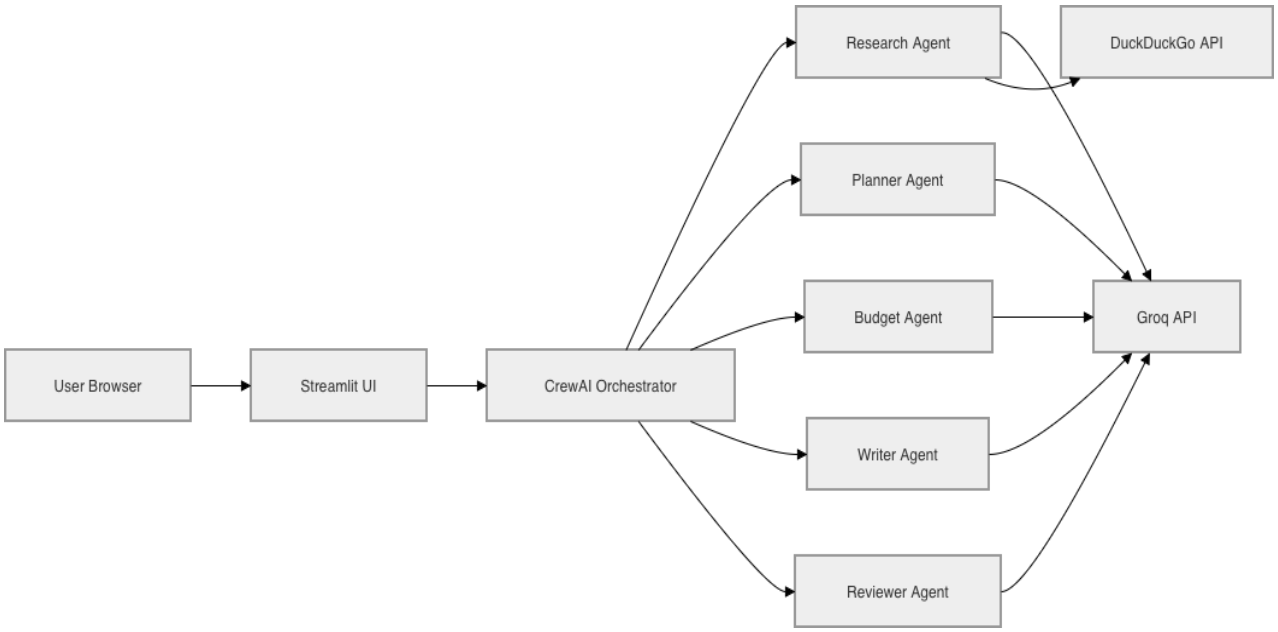
The information flow is fully in-memory and sequential. User input moves from the browser to the Streamlit application, then to the CrewAI orchestrator, and finally across the agents in order. External data is fetched only when required. The final response is returned to the UI without storing user session data in a database.

2.5 Components Design

The main components of the system are:

- **Streamlit UI:** Handles user interaction and result rendering
- **CrewAI Orchestrator:** Controls the execution flow between agents
- **Research Agent:** Collects relevant travel information
- **Planner Agent:** Builds the itinerary structure
- **Budget Agent:** Checks and adjusts plan according to budget
- **Writer Agent:** Converts the output into readable itinerary format
- **Reviewer Agent:** Reviews the generated response
- **Groq LLM API:** Provides language generation capability
- **DuckDuckGo Search API:** Provides external travel-related search data
- **Kubernetes Pod:** Hosts the application container
- **Kubernetes Service:** Exposes the application to users

Diagram 3: Component Diagram



2.6 Key Design Considerations

1. **Stateless Design:** The system does not depend on a database.
2. **Runtime Configuration:** API keys are injected through environment variables and not hardcoded in the image.
3. **Containerization:** The application is packaged as a Docker image and deployed on Kubernetes.
4. **Simplicity:** The architecture is intentionally small and easy to demonstrate in a college project setting.
5. **External Dependency Handling:** The system relies on Groq and DuckDuckGo for intelligence and search features.

2.7 API Catalogue

The system uses the following APIs:

- **Groq LLM API**
  Used for text generation and agent responses. Requires an API key through environment variables.
- **DuckDuckGo Search API**
  Used by the Research Agent to fetch real-world travel-related information. No strict authentication is required.
- **CrewAI Internal Framework**
  Used to manage the sequential handoff between agents. This is part of the application logic rather than an external service.

3. Data Design

3.1 Data Model

The system does not maintain a persistent database. The data model is lightweight and transient. The primary data items are:

- User input: destination, duration, budget
- Intermediate agent outputs
- Final itinerary response
- Application logs

3.2 Data Access Mechanism

All processing happens in memory during a single request flow. Agents exchange data through the CrewAI orchestration pipeline. No permanent data access layer is used.

3.3 Data Retention Policies

User request data is not stored permanently. The only retained artifact mentioned in the design is app_logs.txt, which records system events for debugging and traceability.

3.4 Data Migration

There is no database migration in the current system because no persistent datastore is used. If a database is added in the future, a separate migration strategy will be required.

4. Interfaces

The system has two main interfaces:

- **User Interface:** Streamlit web UI for entering inputs and viewing the itinerary
- **External Service Interfaces:** Groq API and DuckDuckGo API called through HTTP requests

The Kubernetes Service exposes the app interface to the user, while the application container communicates with external APIs over the network.

5. State and Session Management

The application is designed to be stateless. It does not maintain long-lived user sessions or server-side state. Each request is processed independently, and the output is generated from the current user input and the agent pipeline. This makes the system easier to deploy and aligns with the containerized design.

6. Caching

No explicit caching mechanism is defined in the current design. The system processes each request directly through the orchestrator and agents. Since the project is small and stateless, caching is not required at this stage.

7. Non-Functional Requirements

7.1 Security Aspects

- API keys are kept outside the image and passed at runtime.
- The system avoids hardcoding sensitive credentials.
- External communication happens through standard HTTP requests to trusted APIs.

7.2 Performance Aspects

- The app should respond quickly for a single user request flow.
- The containerized setup should keep startup and deployment simple.
- Since the architecture is stateless and lightweight, the system is suitable for small-scale academic deployment.

8. References

1. Docker Official Documentation (Containerization, Networking)
2. CrewAI Official Documentation (Agent Orchestration, Task Definitions)
3. Streamlit Official Documentation (Interactive Web Applications)